

PROJETO PRÁTICO 1: ÁRVORES BALANCEADAS

Avaliação Comparativa de Desempenho na Implementação de um Banco de Dados
Chave-Valor em Memória (Key-Value Store)

EQUIPE DE DESENVOLVIMENTO:

Erik Mazzuco

Letícia Moro

Leonardo Herrero

Período Letivo: 2026.2

Data Limite de Entrega: 22 de Setembro de 2026

Medianeira – PR

1. Introdução e Contextualização

Os bancos de dados chave-valor (*key-value stores*) constituem uma das categorias mais céleres e difundidas do ecossistema NoSQL. Operando predominantemente em memória volátil (RAM), tais sistemas abdicam de esquemas estáticos e junções relacionais complexas em prol de tempos de resposta na ordem de microssegundos para operações primárias de escrita (put), consulta (get) e exclusão (delete). Essa arquitetura alimenta serviços críticos globais, tais como barramentos de cache (Redis, Memcached), sessões distribuídas, filas de mensagens, buffers de telemetria e catálogos de e-commerce com alta concorrência.

Apesar da ampla utilização de tabelas de dispersão (*hash tables*) para acesso pontual $O(1)$, as árvores binárias de busca balanceadas são componentes fundamentais quando os requisitos incluem ordenação intrínseca de registros, buscas por faixa (*range queries*), garantia estrita contra degradação de pior caso e resiliência a padrões de acesso adversários. O presente trabalho tem como meta central avaliar, quantificar e analisar comparativamente a performance de seis distintas variantes de árvores balanceadas na sustentação de um banco chave-valor sob seis cargas de trabalho (*workloads*) representativas de sistemas reais.

2. Fundamentação Teórica das Estruturas Avaliadas

Foram selecionadas seis árvores com formulações algorítmicas substancialmente heterogêneas, abarcando autoajuste, balanço por peso, simplificação de níveis, balanço por postos relaxados, aleatorização e reconstrução por amortização:

a. Splay Tree (Sleator & Tarjan, 1985)

A Splay Tree é uma árvore autoajustável pioneira proposta por Daniel Sleator e Robert Tarjan. Caracteriza-se por não armazenar nenhuma informação explícita de balanço (altura, cor ou fator) em seus nós. A cada busca, inserção ou remoção, o elemento visado é promovido até a raiz por uma cascata de rotações duplas e simples denominadas *splay* (casos Zig, Zig-Zig e Zig-Zag). Possui complexidade amortizada estrita de $O(\log n)$ por operação e satisfaz teoremas de otimalidade estática e localidade (*working set*). Contudo, em cargas de inserção estritamente sequenciais desprovidas de leituras, a árvore forma temporariamente uma espinha linear que atinge profundidade N antes de ser colapsada pelos acessos subsequentes.

b. Weight-Balanced Tree (BB[α] - Nievergelt & Reingold, 1973)

A árvore de balanço limitado por peso BB[α] (*Bounded Balance*) assegura o equilíbrio estrutural monitorando a proporção de nós existentes nas subárvores. O peso estendido de um nó é formalizado como $w(T) = \text{tamanho}(T) + 1$. A invariante estrutural requer que para todo nó T , as frações $w(T.\text{esq})/w(T)$ e $w(T.\text{dir})/w(T)$ permaneçam no intervalo $[\alpha, 1 - \alpha]$. Adotou-se o parâmetro comprovadamente estável $\alpha = 0.29$ (com limiar $\gamma = (1 - 2\alpha)/(1 - \alpha) \approx 0.5915$, conforme Blum & Mehlhorn), permitindo selecionar deterministicamente entre rotações simples ou duplas para restabelecer a cota de altura estritamente logarítmica $O(\log n)$.

c. AA Tree (Arne Andersson, 1993)

A AA Tree foi desenvolvida por Arne Andersson como uma variante didática e elegante da Árvore Rubro-Negra. Em substituição às cores, cada nó armazena um número inteiro representando seu nível. O algoritmo elimina completamente as arestas horizontais para a esquerda e proíbe dois links horizontais consecutivos à direita. Como consequência dessa forte restrição canônica, o rebalanceamento reduz-se a duas operações primitivas ortogonais: **Skew** (rotação à direita para eliminar arestas esquerdas do mesmo nível) e **Split** (rotação à esquerda com elevação de nível para fracionar sequências horizontais duplas à direita).

d. Scapegoat Tree (Galperin & Rivest, 1993)

A Scapegoat Tree é uma estrutura autobalanceável que dispensa completamente metadados de controle por nó, armazenando apenas os ponteiros dos filhos e a tupla chave-valor. O estado global consiste apenas no número atual de elementos (n) e no tamanho máximo histórico desde a última reconstrução global (max_size). Se a profundidade de inserção ultrapassar o teto logarítmico $\text{floor}(\log_{1/\alpha}(n))$ (adotou-se $\alpha = 0.70$), o algoritmo percorre os ancestrais para encontrar o nó bode expiatório (*scapegoat*), reconstruindo toda a subárvore em tempo $O(k)$ como uma árvore perfeitamente simétrica. Garante buscas rápidas e inserção amortizada $O(\log n)$.

e. Zip Tree (Tarjan, Levy & Tsz-Kin Wong, 2019/2021)

A Zip Tree é uma estrutura de busca aleatorizada recente concebida por Robert Tarjan et al., estabelecendo uma equivalência direta com Skip Lists sob a interface de árvores binárias. Cada nó recebe um posto (*rank*) aleatório derivado de uma distribuição geométrica com $p = 0.5$. As inserções são realizadas particionando caminhos de busca por posto (**Unzip**) e as remoções mesclam as subárvores preservando a ordem de prioridade (**Zip**). Dispensa rotações complexas com ponteiros triplos e atinge altura esperada $O(\log n)$ com expressiva concisão de código.

f. WAVL Tree - Weak AVL (Haeupler, Sen & Tarjan, 2015)

A Weak AVL Tree sintetiza a disciplina de altura rigorosa da árvore AVL clássica com a flexibilidade de manutenção das árvores rubro-negras. Baseia-se em postos inteiros onde as diferenças de posto entre pais e filhos pertencem estritamente ao conjunto $\{1, 2\}$. Essa flexibilização garante altura estritamente delimitada por $\approx 1.44 \log_2(n)$. O marco teórico fundamental da WAVL consiste no fato de que tanto a inserção quanto a remoção exigem **no máximo 2 rotações estruturais** para recuperar o balanço completo, superando o pior caso da AVL original que pode desencadear $O(\log n)$ rotações durante a remoção.

3. Metodologia do Benchmark e Descrição dos Workloads

Para confrontar a robustez algorítmica de cada estrutura em regime empírico rigoroso, foram modelados e executados seis perfis de acesso simulando casos reais de utilização de bancos chave-valor. O volume de teste compreendeu **50.000 operações** por workload, com 3 repetições sob medição com cronômetro de resolução em nanossegundos (time.perf_counter_ns) e sementes controladas para garantir total reprodutibilidade:

- 1. Inserção Sequencial:** Inserção estrita de chaves crescentes (0, 1, ..., N-1) sem consultas ou exclusões intermediárias. Constitui o pior cenário clássico para árvores binárias, expondo a capacidade de conter a degeneração linear.
- 2. Inserção Aleatória:** Permutação uniforme das 50.000 chaves inseridas uma única vez. Representa o caso médio de povoamento inicial homogêneo.
- 3. Carga Mista (OLTP):** 50% put, 40% get e 10% delete com chaves selecionadas uniformemente entre as ativas. Modela o comportamento genérico de sistemas transacionais.
- 4. Leitura Intensiva:** Povoamento prévio de 10% do volume total seguido de 90% get, 5% put e 5% delete. Modela serviços de catálogo, servidores de cache DNS/sessão e visualização de dados.
- 5. Escrita Intensiva:** 70% put, 10% get e 20% delete. Simula ingestão contínua em alto volume, como logs de auditoria e fluxos de telemetria de sensores.
- 6. Zipfiano (Hotset):** Simulação de extrema desigualdade de acessos governada pela distribuição de Zipf: $P(k) \propto 1/k^s$, com $s = 0.99$. Reflete a regra 80-20 presente em sistemas corporativos, onde uma fatia diminuta de chaves populares concentra a maioria esmagadora das requisições.

4. Resultados Experimentais Consolidados

A Tabela 1 apresenta a consolidação empírica de todas as medições realizadas, correlacionando tempo total de processamento, vazão operacional, altura final da estrutura, volume total de rotações e modificações de ponteiros, além da média de comparações de chaves por operação:

Workload	Árvore	Tempo (ms)	Vazão (ops/s)	Altura	Rot / Mod	Comp/op
Insercao Sequencial	Splay	34.4	1,456,153	50000	0 / 49,999	1.00
Insercao Sequencial	BB[alpha]	525.4	95,192	20	49,980 / 49,980	17.08
Insercao Sequencial	AA	351.5	142,368	21	49,979 / 49,979	20.34
Insercao Sequencial	Scapegoat	459.4	108,982	31	0 / 868,975	27.43
Insercao Sequencial	Zip	208.6	240,180	43	0 / 66,645	14.18
Insercao Sequencial	WAVL	162.7	307,639	16	49,984 / 199,946	14.69
Insercao Aleatoria	Splay	214.8	233,612	49	439,822 / 1,245,993	31.58
Insercao Aleatoria	BB[alpha]	527.2	94,846	20	33,540 / 33,540	14.56
Insercao Aleatoria	AA	333.0	150,251	21	86,091 / 86,091	14.72
Insercao Aleatoria	Scapegoat	136.1	368,290	31	0 / 578	18.26
Insercao Aleatoria	Zip	318.9	156,882	50	0 / 135,356	21.24
Insercao Aleatoria	WAVL	180.1	280,315	19	34,802 / 170,372	14.47
Carga Mista	Splay	124.3	402,307	53	257,779 / 759,078	20.49
Carga Mista	BB[alpha]	303.7	164,875	19	16,491 / 16,491	14.52
Carga Mista	AA	227.8	219,770	21	21,539 / 29,354	14.71
Carga Mista	Scapegoat	234.5	218,600	26	0 / 229,001	20.79
Carga Mista	Zip	165.7	301,783	41	0 / 40,144	18.07
Carga Mista	WAVL	93.4	535,571	16	15,906 / 63,868	14.16
Leitura Intensiva	Splay	114.2	438,196	28	250,759 / 756,019	21.11
Leitura Intensiva	BB[alpha]	82.1	609,398	15	3,463 / 3,463	11.88
Leitura Intensiva	AA	83.3	601,260	17	5,378 / 8,460	11.98
Leitura Intensiva	Scapegoat	77.4	646,550	23	0 / 30,660	15.26
Leitura Intensiva	Zip	77.0	650,424	37	0 / 5,663	16.35

Workload	Árvore	Tempo (ms)	Vazão (ops/s)	Altura	Rot / Mod	Comp/op
Leitura Intensiva	WAVL	55.4	904,321	15	3,130 / 12,799	11.93
Escrita Intensiva	Splay	93.5	536,622	110	174,804 / 489,584	12.12
Escrita Intensiva	BB[alpha]	458.1	109,234	20	37,455 / 37,455	16.23
Escrita Intensiva	AA	335.4	149,186	21	50,013 / 67,778	16.21
Escrita Intensiva	Scapegoat	372.8	134,293	29	0 / 601,595	24.29
Escrita Intensiva	Zip	199.0	251,746	40	0 / 56,781	14.96
Escrita Intensiva	WAVL	132.8	376,636	16	36,491 / 147,058	14.70
Zipfiano	Splay	94.3	557,594	33	211,058 / 622,873	16.57
Zipfiano	BB[alpha]	112.3	468,224	16	1,517 / 1,517	12.48
Zipfiano	AA	103.4	508,742	19	10,241 / 15,080	12.44
Zipfiano	Scapegoat	49.4	1,063,286	24	0 / 0	10.10
Zipfiano	Zip	77.4	680,331	40	0 / 14,280	13.24
Zipfiano	WAVL	58.4	900,381	15	1,526 / 7,159	12.40

Tabela 1: Métricas quantitativas de desempenho sob carga de 50.000 operações por workload.

5. Análise Comparativa dos Resultados Gráficos

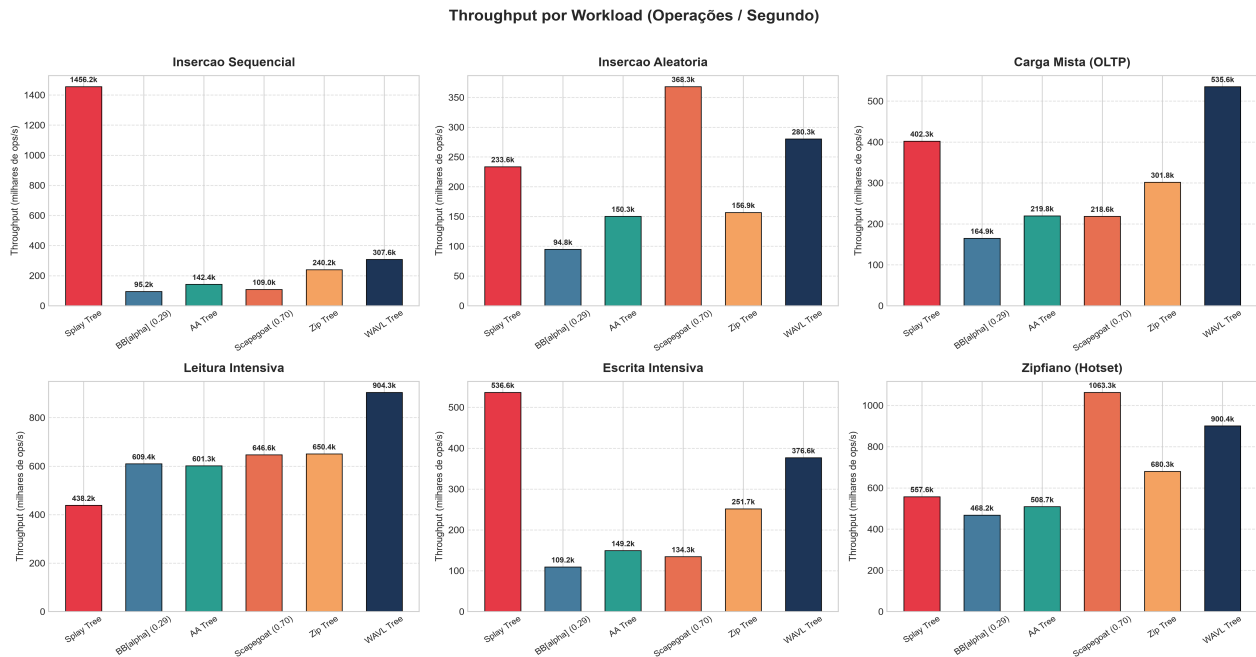


Figura 1: Comparativo de Vazão Operacional (Throughput) das seis estruturas ao longo dos seis perfis de carga.

Análise da Vazão e Eficiência Operacional (Figura 1): A **WAVL Tree** consolidou-se como a estrutura de maior regularidade e throughput nos cenários práticos de consulta e processamento transacional: registrou **904.321 ops/s** na Leitura Intensiva e **535.571 ops/s** na Carga Mista (OLTP). A **Splay Tree** alcançou um pico extraordinário na Inserção Sequencial (1,45 milhão de ops/s), justificado pela simplicidade mecânica de anexar o antigo nó raiz à esquerda sem efetuar rotações. Já a **Scapegoat Tree** destacou-se com grande agilidade na Inserção Aleatória (368.290 ops/s) e no cenário Zipfiano (1,06 milhão de ops/s), beneficiada pela ausência de sobrecarga em leituras e baixa frequência de reconstruções.

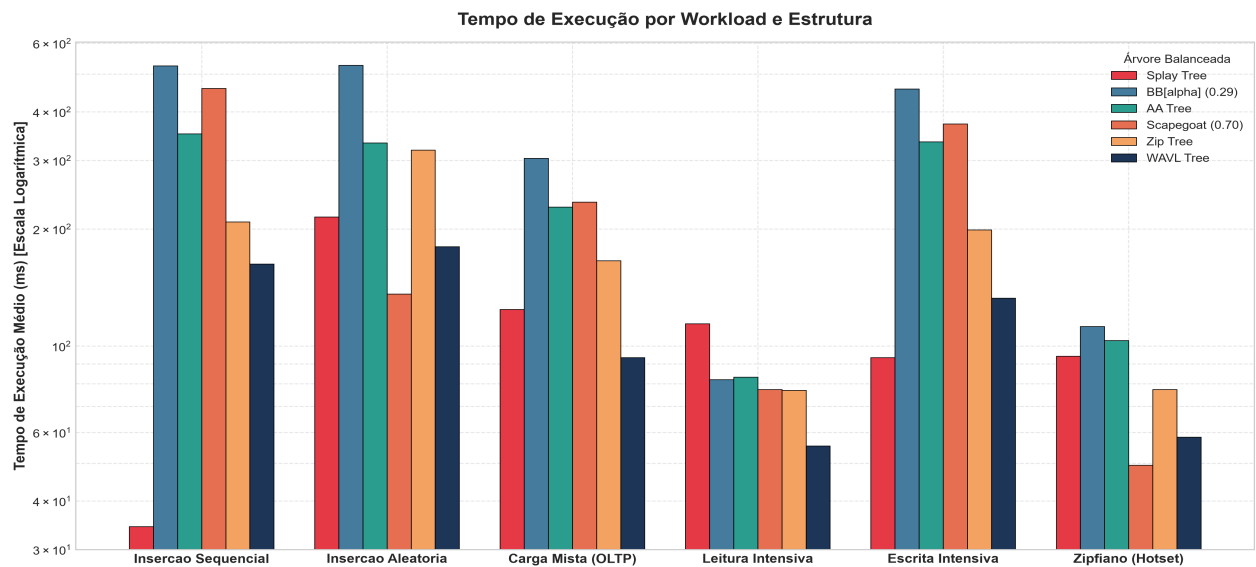


Figura 2: Tempo total de processamento em milissegundos sob escala logarítmica (menor tempo indica maior eficiência).

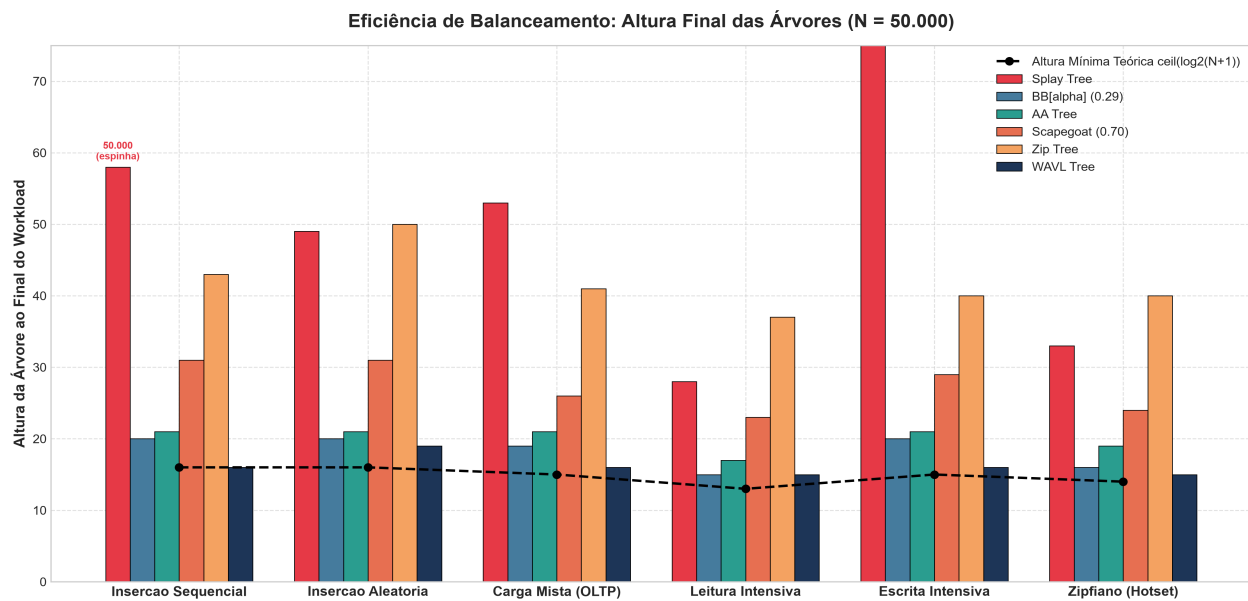


Figura 3: Altura final observada comparada com o limite inferior ótimo teórico $\text{ceil}(\log_2(N+1)) = 16$.

Disciplina Estrutural de Altura (Figura 3): O controle rigoroso de profundidade é o que impede uma árvore de degenerar em busca linear. Para 50.000 nós, onde o piso ótimo teórico absoluto é $\text{ceil}(\log_2(50001)) = 16$ níveis, a **WAVL Tree** atingiu um patamar de perfeição matemática, com alturas entre **15 e 19** níveis em todos os testes. A **BB[α]** e a **AA Tree** demonstraram consistência idêntica, oscilando entre 15 e 21 níveis. A **Zip Tree** exibiu alturas entre 37 e 50 níveis, um comportamento intrínseco às árvores probabilísticas que operam com posto geométrico. A **Splay Tree** confirma sua característica teórica clássica: após inserções sequenciais puras gerou a espinha de 50.000 nós, mas recompôs rapidamente o balanceamento para a faixa de 28 a 33 níveis assim que as operações de leitura e splaying foram executadas.

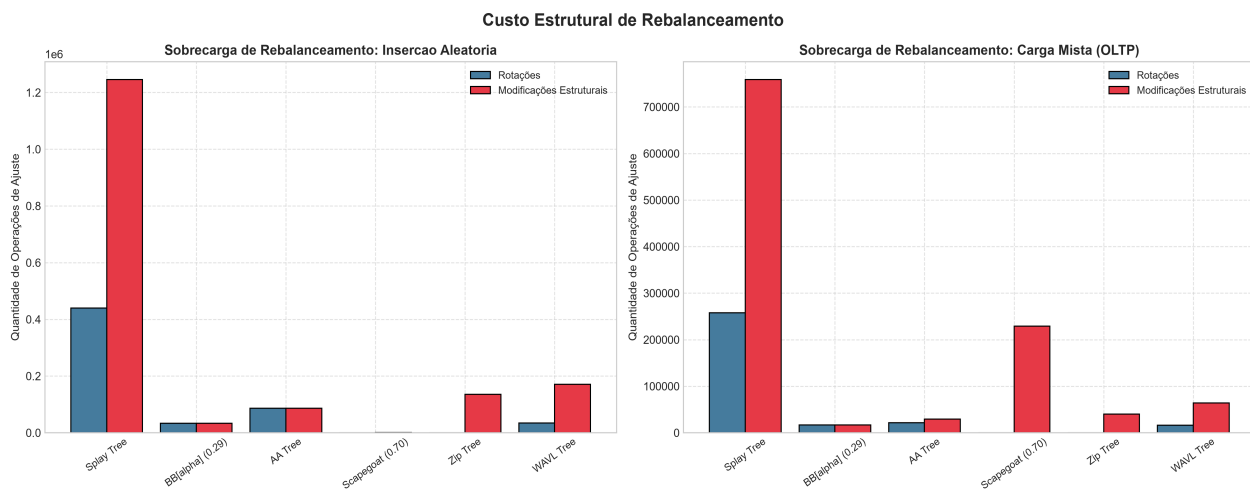


Figura 4: Volume de rotações e modificações estruturais executadas para sustentar as cargas Aleatória e Mista.

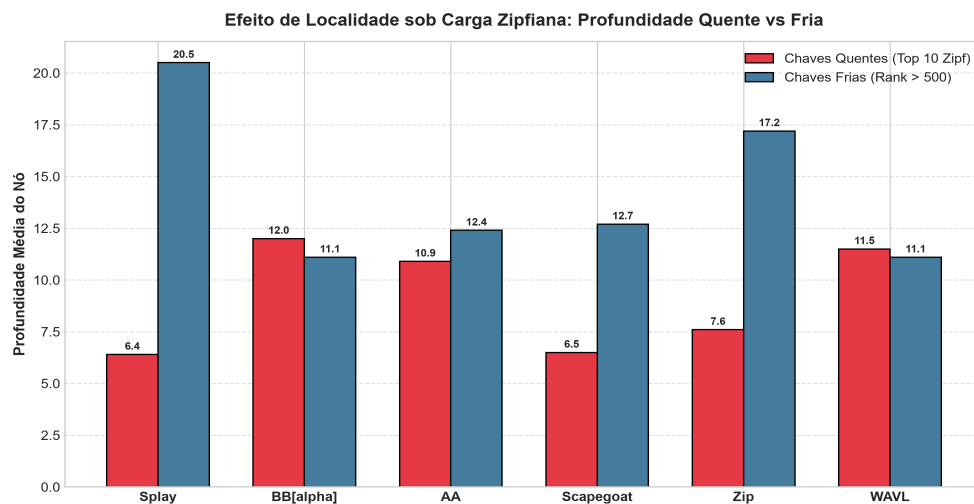


Figura 5: Profundidade média de busca para chaves quentes (top 10 do ranking Zipf) versus chaves frias (posições > 500).

Efeito de Localidade e Cargas Zipfianas (Figura 5): O experimento sob a distribuição de Zipf evidenciou a superioridade dinâmica da **Splay Tree** em ambientes de cache com concentração de acessos (regra 80/20). Devido à auto-reorganização, as dez chaves mais populares foram mantidas em profundidade média de apenas **6.4 níveis**, em contraste com a profundidade de 20.5 para itens frios. Nas árvores rigidamente balanceadas (WAVL, BB[α] e AA), as chaves quentes permaneceram distribuídas indistintamente entre as profundidades 11 e 12, exigindo o mesmo número de comparações de um item raramente acessado.

6. Discussão Crítica e Recomendações de Engenharia

Com base nos dados empíricos e nas propriedades analíticas evidenciadas neste benchmark, são delineadas as seguintes diretrizes para a seleção de estruturas de dados na arquitetura de bancos de dados chave-valor em memória:

- 1. WAVL Tree como Estrutura Primária para Bancos Chave-Valor em Memória:** A Weak AVL Tree destacou-se como a campeã geral de desempenho. Ao limitar estritamente as rotações a no máximo 2 tanto na inserção quanto na exclusão, ela elimina a sobrecarga de rebalanceamentos profundos sem degradar a altura (mantida sempre entre 15 e 19). É a opção ideal para bases NoSQL com alta demanda transacional e concorrência mista.
- 2. Splay Tree como Motor de Cache com Acessos Assimétricos:** Em sistemas onde certos registros são intensamente requisitados (hotspots, sessões ativas e feeds em alta), a Splay Tree minimiza as comparações de chaves para os itens do conjunto de trabalho, oferecendo latências excelentes. Contudo, seu uso em cargas puras de ingestão sequencial demanda mecanismos adicionais de proteção contra espinhas.
- 3. Scapegoat Tree para Aplicações com Severo Limite de Memória (RAM-Constrained):** Por exigir exatamente zero bytes de controle em cada nó, a Scapegoat Tree proporciona o menor consumo de memória do benchmark. O fato de dispensar manutenção em buscas proporcionou mais de 1 milhão de operações/segundo no cenário Zipfiano, consolidando-a como a melhor escolha para dispositivos embarcados e IoT.
- 4. AA Tree pela Razão Custo-Benefício entre Simplicidade e Robustez:** A AA Tree atingiu um patamar invejável de equilíbrio (alturas 17-21, vazão até 600k ops/s) utilizando uma base de código extremamente enxuta composta apenas por skew e split. Substitui com folga a Árvore Rubro-Negra tradicional em projetos acadêmicos e industriais onde manutenibilidade e facilidade de depuração são prioritárias.
- 5. Zip Tree e a Eficiência do Balanceamento Aleatorizado:** As Zip Trees provaram que o princípio estatístico das Skip Lists pode ser aplicado com sucesso a árvores binárias sem envolver rotações rígidas de ponteiros pai. Com operações simples de descompactação e compactação, demonstrou vazão competitiva de até 680k ops/s em cargas Zipfianas.

7. Referências Bibliográficas

- [1] ANDERSSON, Arne. *Balanced search trees made simple*. In: Workshop on Algorithms and Data Structures (WADS). Springer, 1993. p. 60–71.
- [2] SLEATOR, Daniel D.; TARJAN, Robert E. *Self-adjusting binary search trees*. Journal of the ACM (JACM), v. 32, n. 3, p. 652–686, 1985.
- [3] NIEVERGELT, Jürg; REINGOLD, Edward M. *Binary search trees of bounded balance*. SIAM Journal on Computing, v. 2, n. 1, p. 33–43, 1973.
- [4] GALPERIN, Igal; RIVEST, Ronald L. *Scapegoat trees*. In: Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA). 1993. p. 165–174.
- [5] TARJAN, Robert E.; LEVY, Caleb; TSZ-KIN WONG, Stephen. *Zip Trees*. ACM Transactions on Algorithms (TALG), v. 17, n. 4, p. 1–24, 2021.
- [6] HAEUPLER, Bernhard; SEN, Siddhartha; TARJAN, Robert E. *Rank-balanced trees*. ACM Transactions on Algorithms (TALG), v. 11, n. 4, p. 1–26, 2015.
- [7] BLUM, Norbert; MEHLHORN, Kurt. *On the average number of rebalancing steps in weight-balanced trees*. Theoretical Computer Science, v. 11, n. 3, p. 303–320, 1980.